

Procedural Landscape Generation using Artificial Intelligence

Varun Kumar Gupta, B.Tech.

A Dissertation

Presented to the University of Dublin, Trinity College
in partial fulfilment of the requirements for the degree of

**Master of Science in Computer Science (Augmented and
Virtual Reality)**

Supervisor: Dr. Binh-Son Hua

August 2024

Procedural Landscape Generation using Artificial Intelligence

Varun Kumar Gupta, Master of Science in Computer Science
University of Dublin, Trinity College, 2024

Supervisor: Dr. Binh-Son Hua

...ABSTRACT...

Acknowledgments

Thank you Mum & Dad.

VARUN KUMAR GUPTA

University of Dublin, Trinity College
August 2024

Contents

Abstract	i
Acknowledgments	ii
Chapter 1 Introduction	1
1.1 Motivation	4
1.2 Structure & Contents	4
1.3 Summary	5
Chapter 2 Background and Related Work	6
2.1 Background	6
2.1.1 What is Procedural Content Generation?	7
2.1.2 Challenges faced in Procedural Content Generation	9
2.1.3 Use of Deep Learning in Procedural Content Generation?	9
2.1.4 Convolutional Neural Networks (CNNs)	10
2.1.5 Generative Adversarial Networks (GANs)	11
2.1.6 Conditional Generative Adversarial Networks (CGANs)	12
2.2 Stable Diffusion	12
2.2.1 How does Stable Diffusion Work?	12
2.2.2 What is Low Rank Adaptation?	14
2.3 Dataset	15
2.3.1 What are DEMs?	15
2.3.2 What is a heightmap?	16
2.4 Related Work	17
2.4.1 Classical Procedural Methods	17
2.4.2 Simulation-based Methods	17
2.4.3 Example-based Methods	17
2.4.4 Machine Learning	18
2.4.5 Random Generation	18

2.4.6	Interactive Guided Generation	21
2.5	Summary	22
Chapter 3	Methodology	23
3.1	Problem Formulation	23
3.1.1	Identified Challenges	23
3.1.2	Proposed Work	24
3.2	Overview of the Design	24
3.3	Summary	24
Chapter 4	Implementation	25
4.1	Overview of the Solution	25
4.2	Component One	25
4.3	Summary	26
Chapter 5	Evaluation	27
5.1	Experiments	27
5.2	Results	27
5.3	Summary	28
Chapter 6	Conclusions & Future Work	29
6.1	Future Work	29
Bibliography		30

List of Tables

5.1	Variables of the experiment	27
-----	---------------------------------------	----

List of Figures

1.1	Landscapes from famous media	1
1.2	Landscapes from famous media	2
1.3	Heightmap $\implies$ 3D Landscape	3
2.1	Overview of the Iterative Process in PCG	8
2.2	Heightmaps	11
2.3	Heightmaps	16
2.4	Results from [25]	19
2.5	Training Process used in [25]	20
2.6	Architecture for [25]	20
2.7	Architecture for [6]	21
2.8	Results from [6]	22
3.1	Overview of the Iterative Process in PCG	24
5.1	Measurement of System Wakeups	28

Chapter 1

Introduction

Computer Graphics (CG) has been essential for creating stunning, lifelike visuals in various digital media, from video games to movies. CG dates back to the early 1960s with the establishment of digital computers. A key aspect of this is environment art, which captures the story, the context of the characters, and forms backdrops for immersive virtual experiences. May it be the desolated planets of Dune, or lush worlds like Pandora, landscapes give the sense of depth, and world building like nothing else. However, creating realistic terrains is a tedious and complex task.

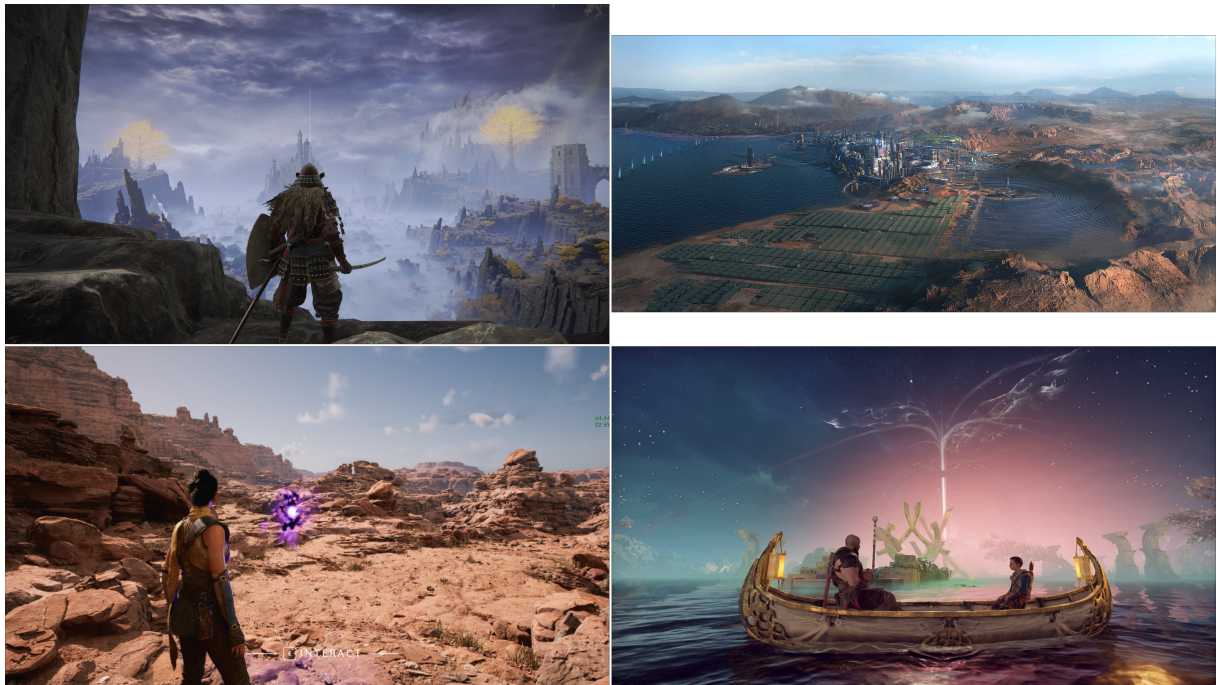


Figure 1.1: Landscapes from famous media

This figure showcases some landscapes from famous games. Top Left - Elden Ring, Top Right - Cyberpunk 2077, Bottom Left - Valley of Ancients, Bottom Right - God of War.

Modelling terrain involves significant manual effort to sculpt and texture vast landscapes accurately. Artists spend hours maintaining consistency across diverse features such as mountains, valleys, rivers, and forests. These challenges highlight the need for more efficient methods.

This is where generative AI (GenAI) and Procedural Content Generation (PCG) come into play. GenAI, especially Deep Learning (DL), has shown incredible potential in creating new content by learning patterns from existing data. In this case, existing heightmaps, which are images that capture the information of a landscape/terrain in a monochrome format. PCG uses algorithms to automatically create complex textures, populate spaces, and structures in a meaningful manner. Both combined offer powerful tools for landscape modelling.

This dissertation integrates Stable Diffusion (SD) in a fast exmaple-based approach to realistic terrain synthesis (Figure 1.2), which is oblivious to various geological phenomena. This method bridges the gap between flexibility and intuitiveness of procedural landscape generation. The aim is not to simulate or model the underlying phenomena, rather learn them and generate *by example*.

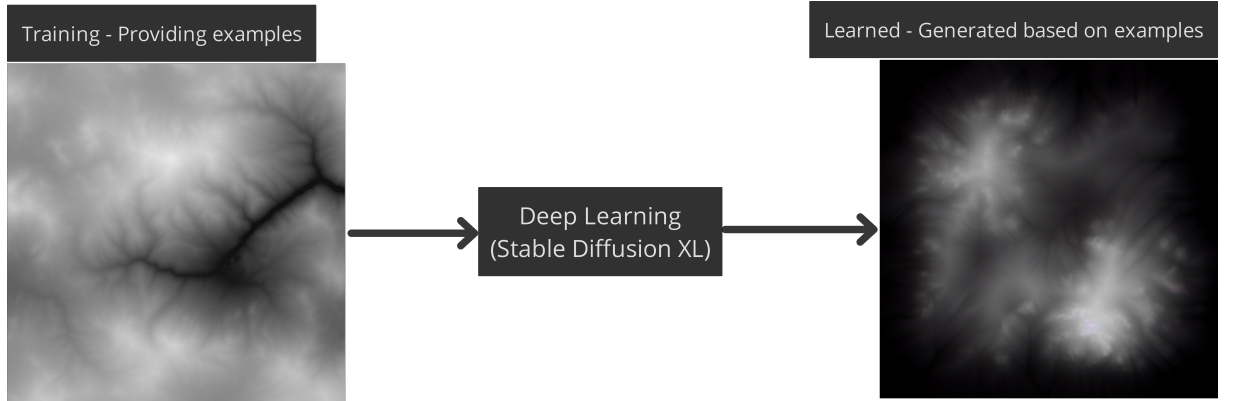


Figure 1.2: Landscapes from famous media

This figure shows the general workflow, where real-world data is feeded to the Stable Diffusion Network, which then learns the important features and generates similar images. The images are generated based on a prompt, which is learned from the training data.

The process involves generating random images that follow patterns of real-world data. In particular, a one-to-one mapping is done:

$$P : Z \implies X \quad (1.1)$$

where Z is random noise and X is the generated image. Due to computational limitations, the generation is tile-based. Where, the overall landscape is divided into chunks,

which are generated keeping in mind the features of the other chunk, or just subdividing the guiding image into smaller parts. To add height to each pixel of the generated image, a one-to-one mapping G , generates a Digital Elevation Model (DEM) for each input tile X . Specifically,

$$G : X \implies Y \quad (1.2)$$

where X is the domain of images produced by P and Y that of DEMs. Both processes follow a *rule-based* approach, where the one-to-one mappings are infinite! The metadata for the generation includes a seed, and hash value that can be used to recreate the results. The detailed process for Stable Diffusion (SD), which is the main DL method of this dissertation, is discussed in more detail in subsequent chapters. The data driven subset of Machine Learning (ML) has proven very useful for many problems, especially in image processing tasks for Computer Vision (CV) [34; 16; 23]. The code for this work is publically available at <https://github.com/VKG5/Procedural-Landscape-Generation>.

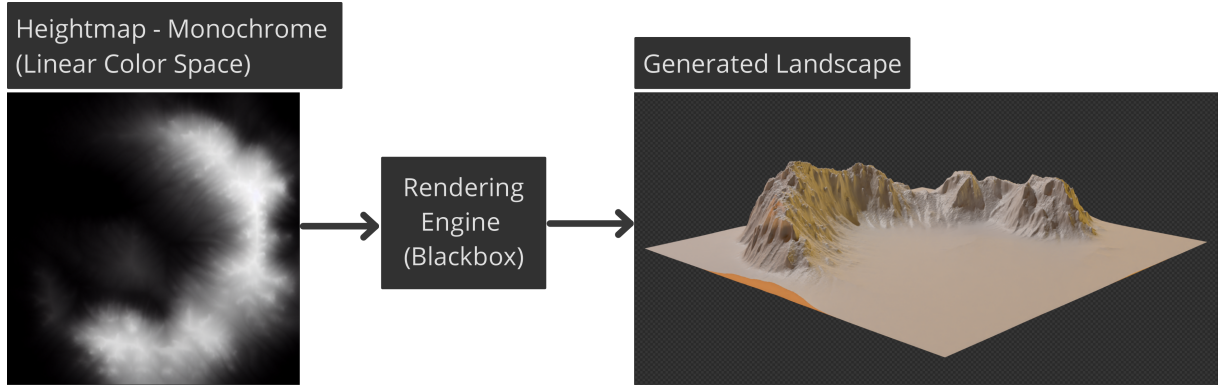


Figure 1.3: Heightmap $\implies$ 3D Landscape

The diagram shows a heightmap, which is a black and white image being converted into a 3D landscape using a rendering engine. Further details about the process will be covered later in the dissertation.

A rendering engine interprets the data from a given heightmap; generates a mesh: vertices, indices, UV Map, and Normals; textures for the landscape, which can be the heightmap itself; and lighting. The engine is responsible for converting a simple heightmap into detailed 3D terrains. This facility is available in all modern graphics software. The mesh must be highly subdivided (huge number of vertices) for best results. For the thesis, a custom engine has been developed to maintain speed and controllability.

1.1 Motivation

”Ban AI art” is a movement against AI-generated content, reflecting concerns within the artistic community. Personally, the mantra ”Artists first, AI later” drives my motivation to develop tools, create art, and craft immersive virtual experiences. PCG should be a tool in the hands of an artist, not a replacement for the artist. It should serve as a starting point or a supportive element, enhancing the creative process without outweighing the human efforts.

The virtual experiences that engross audiences are born from the efforts of hundreds of dedicated individuals working on a project for years. The emotional resonance of these experiences, stems from the careful curation by directors, producers, artists, and technical artists. PCG and AI have the potential to support small teams and hobbyists, to create content-rich games through automation and rapid iterations. Allowing them to focus on creative aspects while retaining full ownership of the games, even without the extensive resources of a large company [37].

The main aim is to develop a set of tools that would allow hobbyists, companies and the the Open Source Community to generate landscapes without a steep learning curve. Often, associated with tools like World Machine, Gaea, even node-based workflows such as Blender Geometry Nodes, Houdini, and Substance Designer. Simplifying the process, users can focus on creativity and innovation without being obstructed by software intricacies.

1.2 Structure & Contents

This thesis is structured as a series of chapters, organized to introduce both the theory and methods followed by their practical application. This structure is motivated by the interdisciplinary nature of PCG research and practice, where algorithms and techniques used come from various fields.

Chapter 2 reviews previous research in the field of Procedural Landscape Generation using AI, focusing on methods involving Generative Adversarial Networks (GANs), Conditional Generative Adversarial Networks (CGANs), Patch GANs, and Stable Diffusion (SD). The chapter also provides an introduction to the terminology required for understanding the thesis. Diving into contemporary industry methods, including simple fractal algorithms and other noise-based approaches.

Chapter 3 outlines the methodology employed for this thesis, detailing the problem addressed, the solutions implemented, and the overall design of the application.

Chapter 4 builds on Chapter 3, explaining the application’s details, covering the

architecture of Stable Diffusion models, and exploring some computer graphics fundamentals, such as Vertex Array Objects (VAOs) and the rendering pipeline.

Chapter 5 discusses the evaluation of the generated landscape's quality once it has been implemented.

Chapter 6 addresses the caveats and shortcomings of this thesis and suggests possible improvements for future research.

1.3 Summary

This dissertation explores how Deep Learning (DL), particularly Stable Diffusion and Procedural Content Generation (PCG) can be combined to make terrain modelling easier and more effective. The aim is to develop a technique that simplifies the creation of detailed and realistic landscapes, while retaining a certain degree of control with the user. This research contributes to the field of Computer Graphics (CG) by presenting new methods for automated landscape generation, enhancing efficiency and quality.

Chapter 2

Background and Related Work

This chapter covers the essential terminology and knowledge required to make sense of the thesis. Includes a section about related work that discusses the advances made by previous researchers in the field. By understanding key concepts such as Procedural Content Generation (PCG), Generative Adversarial Networks (GAN), Conditional GANs (CGANs), Patch GANs and Stable Diffusion (SD), the reader can better appreciate the context and significance of this research. This foundation sets the stage for a comprehensive exploration of AI-driver procedural landscape generation.

2.1 Background

Despite more than thirty five years of research in terrain modelling, creating virtual terrains using contemporary techniques remains a demanding task [6]. One reason for this is the variance in the geographical requirements of different media. They range from floating mountains, dead deserts, to underwater cities, and sometimes even combinations of completely opposite biomes. In real-life, terrains are exposed to years of different erosion and land-forming agents such as hydrosion, vegetation, varying temperatures, that are difficult to express mathematically as algorithms. This poses many challenges, including expressing the user’s intent, while not overwhelming them with parameters and controls.

Most existing terrain modelling techniques can be categorised as procedural 2.4.1, simulation-based 2.4.2, and example-based 2.4.3. The leading industry software, including Gaea and World Machine [27; 30], uses simulation and fractal-based approaches. Many terrains have strong fractal features, where PCG allows for fast terrain generation, but they often fail to provide control over said features.

Simulation-based methods, e.g., hydrosion and erosion, generate geologically correct models, but lack user control and are computationally expensive. Sketch-based methods

provide a high level of control, but do not generate accurate terrains, often skipping on details, and editing large terrains is very tedious and slow. Existing example-based methods often generate repetitive results and have a hard time generating new features. One common drawback of these existing algorithms is that they cannot be applied to large-scale terrains.

With the increase in computational power, and new hardware, such as the Nvidia Ray Tracing Texel eXtreme (RTX) the demand for larger open world games is more than ever before. Not only games but also other media, such as movies, TV shows, and almost all digital media that involve Computer Graphics (CG) at a certain level.

2.1.1 What is Procedural Content Generation?

Procedural Content Generation (PCG), is the *algorithmic creation of 3D content with limited or indirect user input* [37; 22]. Alternatively, PCG can be defined as the automated creation of content by computer software, which may involve minimal or no human input during the generation process.

”Content” in this context refers to various elements such as levels, game rules, textures, materials, maps, missions, items, and more. However, Game Engine and Non-Player Character (NPC) behavior or NPC AI are generally not considered content. This distinction arises because more extensive research in Artificial Intelligence (AI) and Computational Intelligence (CI) focuses on character behaviour rather than procedural methods. While one may argue that PCG derives from AI methods, it is distinct from concepts such as Reinforcement Learning (RL), where AI learns how to play a game. Some prefer the term ”generative methods” for a broader category that includes PCG [14]

A *PCG method* functions as a component of a larger *PCG System*. Below are some notable examples of PCG applications:

- **Planet Generation in Games:** A tool that generates new planets and populates them with flora and fauna without human input. A prominent example is *No Man’s Sky*, where each new campaign starts with a procedurally generated galaxy featuring unique planets and weather conditions [35].
- **Landscape Generation Software:** Advanced algorithms generate highly detailed and realistic landscapes. Software like *Gaea* and *World Machine* use node-based workflows, allowing users to create complex terrain features through a node graph. This provides control over each generation step, including erosion, sedimentation, and tectonic activities, positioning them at the forefront of PCG in landscape generation [30; 27].

- **Game World Generation:** In games such as *Civilization 6*, players can choose the general shape of the map by selecting a world generation algorithm (e.g., Continents, Archipelago, or Pangaea). The computer then generates and evaluates the terrain in a predictable manner and populates resources and enemy AI factions accordingly [33].
- **Procedural Animations:** In *Rain World*, procedural animations minimize human interactions and resources. The player character, a *Slugcat*, and enemies like Long Legs and Lizards react to the surroundings using a full-body IK system instead of pre-made animations. This enhances immersion by generating unique animations based on the environment and player-computer interactions.
- **Procedural Foliage:** In *Horizon Zero Dawn*, world data feeds into an algorithm that determines the distribution of various types of foliage. This method not only makes the world appear more natural and lush but also enhances the overall experience [38; 40].

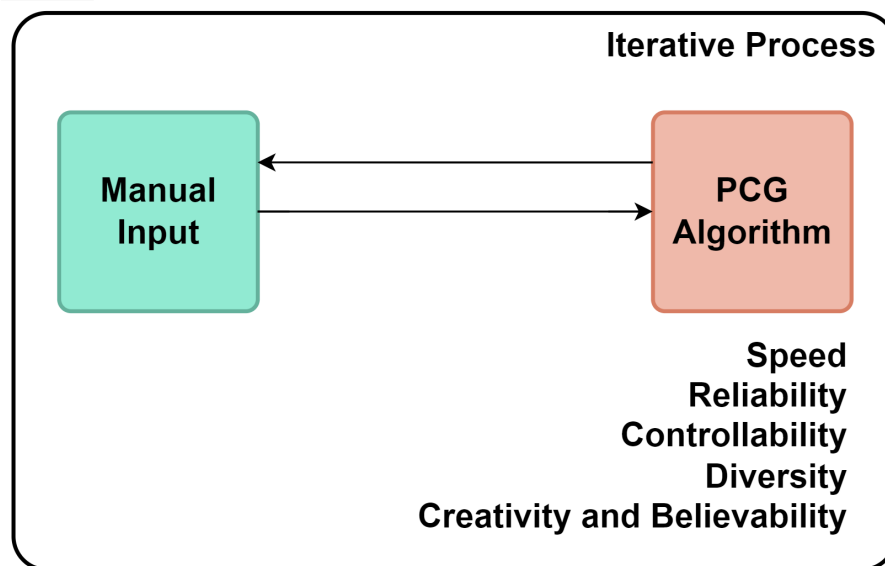


Figure 2.1: Overview of the Iterative Process in PCG

The diagram illustrates interaction between manual input and the PCG algorithm. The process starts with manual input provided by human creators, which feeds into the PCG algorithm. This algorithm generates content based on the input and iteratively refines it. Key benefits of this process include speed, reliability, controllability, diversity, and enhanced creativity and believability of the generated content.

2.1.2 Challenges faced in Procedural Content Generation

Major constraints in PCG include trade-offs, such as between speed and quality, or expressivity/diversity and reliability. Figure 2.1 illustrates various desirable properties of PCG solutions, some of which are outlined below:

- ***Speed:*** This can range from milliseconds to months, depending on whether the content is generated during runtime or development.
- ***Reliability:*** The extent to which the algorithm adheres to specified criteria and produces usable results. For example, a building without exits and windows is undesirable.
- ***Controllability:*** As stated earlier, PCG is intended to be a supportive tool, meaning there should be controllable parameters that guide the generation. For instance, generating a landscape from a black-and-white sketch that defines regions for mountains, rivers, lakes, etc.
- ***Expressivity and Diversity:*** The content should exhibit sufficient diversity or variation. For example, if there are only minor variations on a repetitive theme, or a level generator produces the same type of levels each time. Achieving diverse content while maintaining quality is a challenging task.
- ***Creativity and Believability:*** The generated content should not appear as though it was created by a computer. There should be elements of human imperfection.

Deep Learning (DL) in combination with PCG addresses many of these challenges. One significant advantage is speed; contemporary PCG methods can take hours to generate a single landscape, but with DL, it can be achieved within a few seconds. Additionally, because DL models **can** be trained on real-world data, the generated results are reliable and often have a certain level of quality. The use of DL also allows for extensive control over diversity and creativity, as these can be manipulated by adjusting the input data and parameters of the model. This flexibility enables the generation of varied and creative content, tailored to specific needs or artistic visions.

2.1.3 Use of Deep Learning in Procedural Content Generation?

Procedural Content Generation (PCG) continues to evolve, leveraging advances in hardware and AI to create more complex and diverse content. Recent implementations extend beyond level generation, maps, and textures to include dynamic narratives, complex

ecosystems, and interactive storylines. Large Language Models (LLMs) such as ChatGPT can create branching storylines and character dialogues that respond to player choices, enhancing the narrative depth of games.

Modern PCG techniques utilise Deep Learning (DL) algorithms to learn from extensive datasets, enabling the generation of content that closely mimics professional data in the real world. This approach has proven effective in creating realistic terrains and player-adaptive worlds. By analysing a user's play style and preferences, PCG systems can generate features and content that align with the individual's play style, increasing replayability and immersion on a personal level [25; 22; 37].

Beyond gaming, PCG is increasingly used in fields such as real-time Visual Effects (VFX), Virtual Reality (VR), Simulation Training, and Urban Planning. Entire cities, landscapes, crowds, and infrastructure models can be generated for simulation and analytical purposes. This allows for rapid iteration of models for guns, vehicles, characters, and more, providing a significant starting point instead of starting from scratch each time. For instance, concept art can be transformed into a 3D model using 3D Generative AI models like *Luma AI - Genie*, which can then be further refined. Even rough sketches can generate concept art, demonstrating the versatility of these tools.

In general, DL complements PCG, solidifying its role as a vital tool in creating rich, immersive, and personalised virtual experiences.

2.1.4 Convolutional Neural Networks (CNNs)

A CNN is a deep learning network consisting of an input layer, multiple hidden layers, and an output layer. CNNs are inspired by the connectivity patterns in the human mind [36; 2]. They consist of neurons, perceptrons and many similar terminologies and have multiple layers. The first layer typically expects an image as input, which is then convolved upon in each subsequent layer. The hidden layers process the input by performing a dot product with a kernel consisting of trainable weights [25]. The result is passed into a pooling layer which reduces the input dimensions for the next layer. The fully connected layer is responsible for classifying images based on features extracted in previous layers. The term *fully connected* means that each neuron in one layer is connected to each neuron in subsequent layer [36]. The output layer computes the error of the generated output with respect to the ground truth. Then it passes back the computed error to the previous layer, updating the trainable weights accordingly.

CNNs are way faster than generic feedforward neural networks and take less connections, and parameters, thus, giving faster generation times.

2.1.5 Generative Adversarial Networks (GANs)

GANs is a deep learning architecture. It trains 2 neural networks, which then compete against each other to generate new believable data from the provided dataset [31; 10]. The 2 networks are called - *Generator G* and *Discriminator D* respectively. As the name suggests, the generator network generates the image from randomly sampled noise. The discriminator is responsible for computing the similarity, or believability, of the generated image. The generator is optimised over time to reproduce the data distribution p_{data} , generating images (or any data) that are difficult for the discriminator to differentiate from real-world sample data p_{data} . Their training procedure is a *minmax two-player game* [25; 10].

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))] \quad (2.1)$$

where z is a random noise vector, chosen from previously sampled noise p_z . Often, it is a normal or uniform distribution. x is a real image, from data p_{data} . [10] prove that given enough capacity, the generator can learn to replicate the true data distribution.

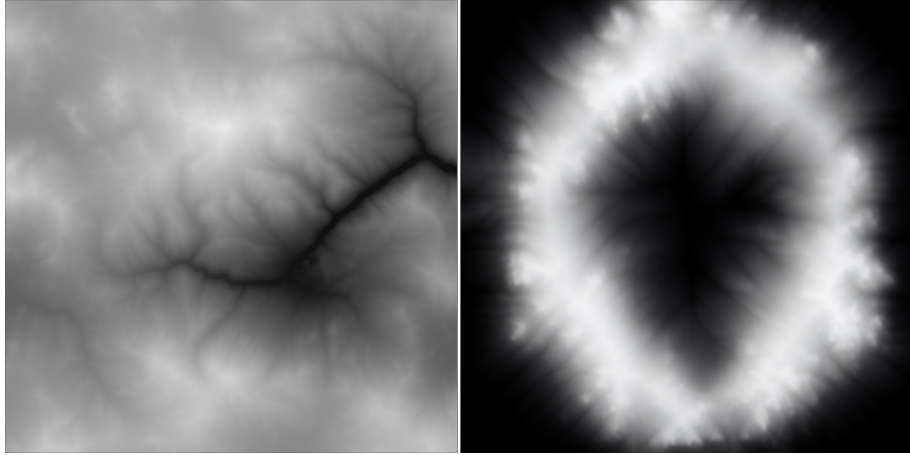


Figure 2.2: Heightmaps

The diagram shows 2 heightmaps, the left one is generated using Gaea and the right one using Stable Diffusion, as a part of this dissertation, which is discussed more in later chapters. The left heightmap is of a low mountain-grassland area, with a river network running through the top left, whereas the right one is high mountains, rocky with a small lake in the center. The left image took hours to develop using a node-based workflow, whereas the right one took 6-7 seconds.

2.1.6 Conditional Generative Adversarial Networks (CGANs)

First examined in [19], CGANs extend GANs by adding additional information as input. This can be a class label, or in this case, extracted features. This makes the discriminator and generator biased towards the particular label. Denoting the additional conditional variable as c , 2.1 can be modified as:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x|c)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z|c)|c))] \quad (2.2)$$

GANs and CGANs have been the center of research for Procedural Landscape Generation prior to Stable Diffusion. Although the results are good, there are limitations, including the quantity and diversity of training data. GANs are notorious for large training datasets. According to NVIDIA, the ideal GAN is trained with 50,000 to 100,000 images. Additionally, GANs struggle with generating high-resolution images, maintaining consistency among adjacent tiles, which causes loss of realism and less varied outputs compared to ones generated using Stable Diffusion.

2.2 Stable Diffusion

Stable Diffusion (SD) is a Generative Artificial Intelligence (GenAI) model that can produce realistic images based on a text prompt. Originally launched in 2022, the model's usecases have expanded to video and animation generation. SD is **not** a Convolutional Neural Network (CNN), although it uses CNN as part of its architecture. It is a model based on diffusion technology which generates images by progressively denoising a randomly sampled noise. It uses a latent space that reduces processing requirements, making it accessible on consumer systems. GANs 2.1.5, could not do this and had large computational requirements, making it harder for the general public to use. SD models can be fine tuned to a specific purpose, in this case, heightmap generation through transfer learning. The ideal dataset contains about 20-100 images, which is astronomically lower compared to GANs.

2.2.1 How does Stable Diffusion Work?

Diffusion models differ from other contemporary image generation models. They use a Gaussian noise to encode an image. Then, using a noise predictor together with a reverse diffusion process to recreate the image [32]. Stable Diffusion consists of 3 parts: Variational Autoencoders (VAE), U-Net, and an optional text encoder [34].

- **VAE:** It compresses the image from pixel space to a smaller latent space, capturing the key features of the image. This process involves gradually adding noise to the training data, in this case heightmaps, and learning to reverse the process [34]. The model learns to predict the noise and generate images by denoising random noise. The forward process of adding noise can be described as:

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{\alpha_t}x_{t-1}, (1 - \alpha_t)I) \quad (2.3)$$

where x_t is the image at time t , α_t is a hyperparameter controlling the noise level, and $\mathcal{N}$ denotes a normal distribution.

The complete forward process over T steps:

$$q(x_{1:T}|x_0) = \prod_{t=1}^T q(x_t|x_{t-1}) \quad (2.4)$$

The reverse process can be described as:

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t)) \quad (2.5)$$

where μ_θ and Σ_θ are the mean and variance predicted by the neural network.

- **U-Net:** At core, this is a CNN. This block is composed of a ResNet [12], denoises the output from forward diffusion backwards to obtain a latent representation. It consists of an encoder (downsampling) and a decoder (upsampling) with skipped connections between corresponding layers. This helps in capturing both the global and local features of the image.

The model is trained to minimise the difference between the noise added during the forward process and the noise predicted by the neural network. This can be represented as:

$$L(\theta) = \mathbb{E}_{t, x_0, \epsilon} [\|\epsilon - \epsilon_\theta(x_t, t)\|^2] \quad (2.6)$$

where ϵ is the noise added in the forward process, ϵ_θ is the predicted noise, and x_t is the noisy image at time t .

- **VAE Decoder:** It generates the image by converting the latent representation back into pixel space.

The denoising step can be conditioned using text, an image, or other data. For this thesis, text and image are supported. The conditioning data is encoded and exposed to the denoising U-Nets via the cross-attention mechanism [1]. For conditioning text, a pretrained text encoder is used to transform text prompts to an embedding space.

- **Cross Attention:** is a machine learning-based **attention** method that simulates how human attention works. It gives different levels of importance to different components of a sequence. In Natural Language Processing (NLP), this refers to assigning different priorities to different words in a sentence. Fed to the U-Net, the model focuses on important parts when generating an output.

- Q : Query matrix (from the decoder)
- K : Key matrix (from the encoder)
- V : Value matrix (from the encoder)
- d_k : Dimension of the key vectors

The cross-attention mechanism is defined by the following equation:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (2.7)$$

Here, the query matrix Q is multiplied by the transpose of the key matrix K^t . The resulting scores are scaled by the square root of the dimension of the key vectors d_k . These scores are passed through a Softmax function get the *attention weights*. This clamps the weights to a $(0, 1)$ range. The attention weights are then used to calculate the weighted sum of the value matrix V .

Studies point to increased computational efficiency for training and generation as an advantage of latent diffusion models (LDMs) [28].

2.2.2 What is Low Rank Adaptation?

Low Rank Adaptation (LoRA), is a lightweight training technique used to fine-tune Stable Diffusion (SD) models without the need to train a new model. Proposed by Microsoft researchers. Models like SD have billions of parameters. A good example is GPT-3 175B; deploying and redeploying independent instances of the model is not feasible [13]. The training is both expensive and time-consuming.

LoRAs freeze the pre-trained model weights and inject trainable rank decomposition matrices into each layer of the Transformer architecture [13]. This greatly reduces the

number of trainable parameters, allowing for faster training times and more manageable file sizes (usually 2MB to 500MB). This makes LoRA models easier to store, share, and use on consumer GPUs, making them more accessible.

Suppose that $W_0 \in \mathbb{R}^{d \times k}$ represents a pre-trained weight matrix of size $\mathbb{R}^{d \times k}$. Where, d is the number of rows and k is the number of columns; $d, k \in \mathbb{R}$. It changes by ΔW (the **update matrix**) such that the fine-tuned model's weight are:

$$W' = W_0 + \Delta W \quad (2.8)$$

LoRA uses a technique that reduces the rank of this update matrix ΔW by rank decomposition, such that:

$$\Delta W = B \times A \quad (2.9)$$

where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$, such that $r \ll \min(k, d)$.

By constraining W_0 , computation and memory is saved. There is an iterative fine tuning of A and B , which contain the trainable parameters for application. The results in the fine-tuned model's forward pass look like this:

$$h = W'x = W_0x + BAx \quad (2.10)$$

For this thesis, it is sufficient to apply rank decomposition to cross-attention layers which are responsible for integrating the prompt and image information.

2.3 Dataset

The dataset for this dissertation is taken from Opentopography, some of which is also generated using Blender [24; 4]. OpenTopography provides high-resolution topography data and tools, particularly NASA Digital Elevation Models (DEMs) which represent the Earth as a heightmap.

2.3.1 What are DEMs?

Digital topography is often expressed in the form of Digital Elevation Models (DEMs). It is a widely accepted format that includes scientific research, digital media, and more. Data producers are also spread over various industries, making it crucial. DEMs provide fundamental depictions of the three-dimensional shape of the Earth's surface and are useful for a variety of disciplines [11]. The various physical phenomena are represented as "spheres". There are 6 spheres that have surfaces or boundary layers along which they

interface with their surroundings. They are referred to as "real surfaces" [7]. The real surfaces are referenced by elevation since they are too complex for rigorous mathematical algorithms. Simplifying this, they are mathematically represented as "topographic" surfaces. A DEM represents the terrain, with no assumptions about its mathematical properties [11].

2.3.2 What is a heightmap?

Heightmap, also known as a heightfield, is a greyscale image with the texel value corresponding to the distance by which a vertex should be moved along its normal. In order to have highly detailed terrain it is necessary to have a high resolution mesh. Figure 2.3 shows 2 heightmaps, lighter shades represent higher elevations, while darker shades indicate lower elevations.

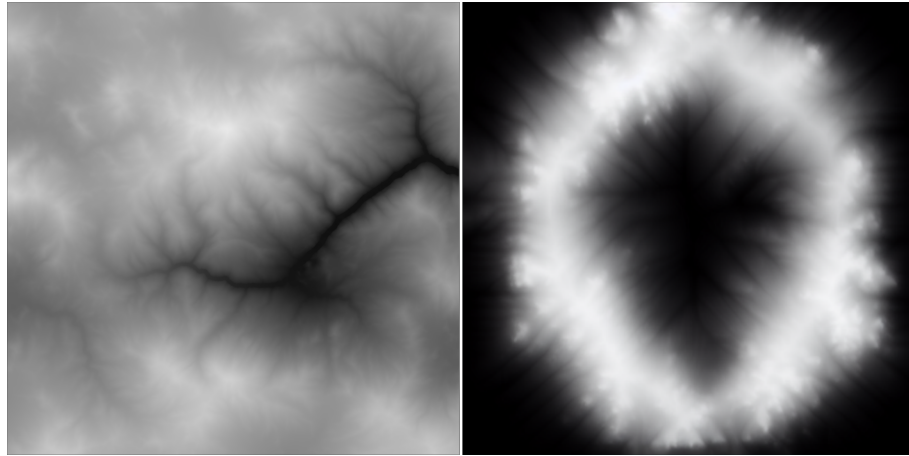


Figure 2.3: Heightmaps

The diagram shows 2 heightmaps, the left one is generated using Gaea and the right one using Stable Diffusion, as a part of this dissertation, which is discussed more in later chapters. The left heightmap is of a low mountain-grassland area, with a river network running through the top left, whereas the right one is high mountains, rocky with a small lake in the center. The left image took hours to develop using a node-based workflow, whereas the right one took 6-7 seconds.

- **Advantages:** They are computationally efficient, as they are simple to compute and manipulate. Allowing for easier modification, and heightmaps are compatible with almost all 3D rendering software, making them a universal way of sharing data across pipelines.

2.4 Related Work

Landscape modelling is a huge part of the graphics industry. The reader is referred to [22] for an overview of Procedural Content Generation and to [25; 6] for a brief on Procedural Landscape modelling using GANs.

2.4.1 Classical Procedural Methods

These are computationally efficient, because they use noise, often times fractal/perlin, that visually resembles a real terrain. Fractals are used for terrains that are self-similar, or have an obvious repeating pattern, and can be observed as fresh and uneroded [8]. Fractals can also be used to define river networks and models generated by Lindenmayer Systems (L-systems) [17]. Later, fractals were used to generate fresh (not eroded) terrains restricted by user-described river trajectories [3]. The biggest limitation in procedural methods is their control. Fractals are difficult to predict [6]. There has been more research in this field, and the control has improved. Recently, [9] introduced a hierarchical distribution tree that models terrains as a distribution of primitives and procedurally blends them together. They are carved, blended and warped to make a consistent generation. Another factor that should be taken into account is that landscapes are never in their **pure fractal form**, as they are always subject to physical phenomena, the most prominent being erosion, followed by hydrosion.

2.4.2 Simulation-based Methods

These were introduced by [20], who used various kinds of erosion algorithms to modify fractal terrains. Heightfields are the most commonly used type of data for erosion simulation, including generations through *noise synthesis*. Later, they were extended using hydraulic erosion by [21]. This focused on generating mountains and valleys using river flow, erosion, and various weathering effects. Although layered data structures, volumetric data, and smooth particle hydrodynamics are also used, their use-case is more towards physics simulations. While simulation-based algorithms generate geologically correct models, they are difficult to control and computationally demanding. It was only possible to run these algorithms at an interactive frame rate by using GPUs. Expressing the user's intent is also difficult using this method.

2.4.3 Example-based Methods

These use existing terrains or user sketches to define terrain models. One of the earlier examples is from [39], which uses heightfield patches from real terrains and combines

them into user-defined sketches. These methods are often interactive and almost real-time. Heightfields also have multiple uses, including luminance-based texture generation, roughness maps, and more toward a Physically Based Rendering (PBR) pipeline. Sketch-based approaches can generate terrains that have high control, but may be geologically incorrect. Hybrid approaches combine this with simulation-based methods that produce better results but are limited to very small terrains.

2.4.4 Machine Learning

ML has been used to generate images, 3D experiences (NeRF, Gaussian Splatting), 3D models, and other high-dimensional data similar to the training set. Texture synthesis [25] to generate images of a given type [6], to predict future frames in games and movies [18], to complete procedural buildings [?]. Neural networks learn a one-to-one mapping between input and output, often using convolution. These models are limited by the optimisation required during the decoding stage, which results in low-order interactions between output variables. U-nets map input to output by first decreasing the spatial resolution iteratively and then restoring spatial resolution with upsampling. They benefit from additional skip-connection layers among the same resolution.

2.4.5 Random Generation

Previously proposed methods using Generative Adversarial Networks (GANs) are also stochastic in nature [6; 25]. In particular, in [25], the authors have divided the process into sequential GANs. Firstly, they are generating random images that mimic the real satellite images of the dataset. Images of size 256x256 are generated, using a technique of progressive growing GANs (ProGANs) introduced in [15]. This architecture divides the training into multiple stages. Instead of training all layers of the generator and discriminator models, ProGANs are trained one layer at a time. This leads to exponential growth of the generated images on every step. The method stabilises the training process and reduces the duration, leading the generator to convergence and producing high-resolution images simultaneously. Adding more layers increases the quality of the generated images exponentially. Figure 2.5

The weights for all previous layers remain trainable, and new layers are *faded in* gradually. The process of fading is controlled by a parameter $\alpha \in [0, 1]$ over multiple iterations. The discriminator is a symmetrical copy of the generator. Input images are either "fake" images generated by the generator or real images from the dataset. Input images are downsampled to the current training resolution through a series of convolutional layers, until a boolean decision is returned.

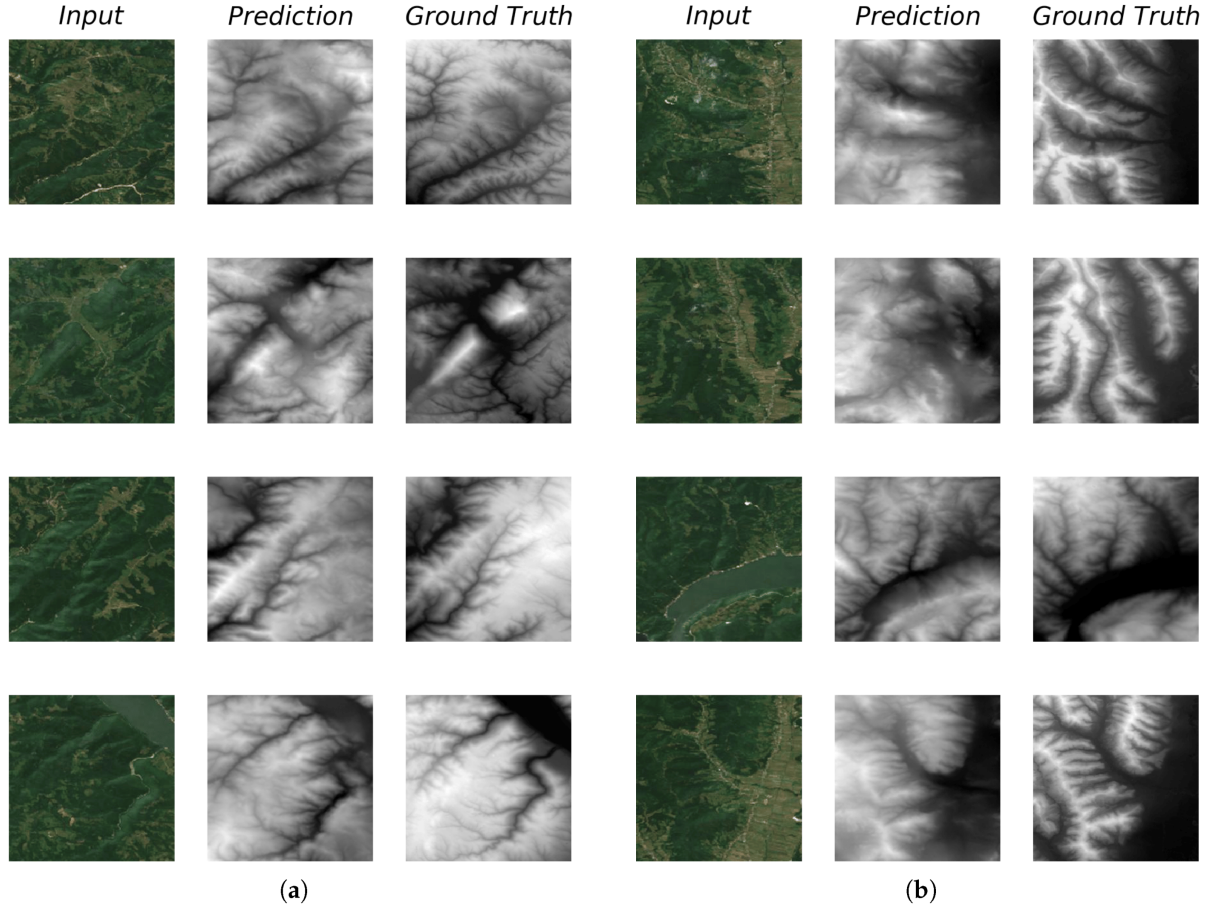


Figure 2.4: Results from [25]

The images above show the generators results on previously unseen data. As visible, the results are good, with the model being able to convert given satellite images to DEMs.

Even when the mapping is not precise, it gives a general representation of the DEM.

Following this, the authors used a pix2pix architecture to train a CGAN [25]. They have used an encoder-decoder architecture, a U-net as described in [29]. The process followed is similar as above, with the addition of skip-connection layers. A major difference compared to traditional CNN architectures is the decision based on an entire image. [26] uses PatchGAN which maps the 256x256 image to a square array of outputs, in which each output "pixel" signifies whether the corresponding patch is real or fake. The final output is the average of all predicted patches. Combining the generator and discriminator, the U-net and PatchGAN respectively, the authors get good results for complex tasks. Figure 2.4 shows the results achieved in this paper.

An overview of the whole architecture can be found described in Figure 2.6. A typical GAN architecture has been used with various modifications as stated above in the respective generators and discriminators.

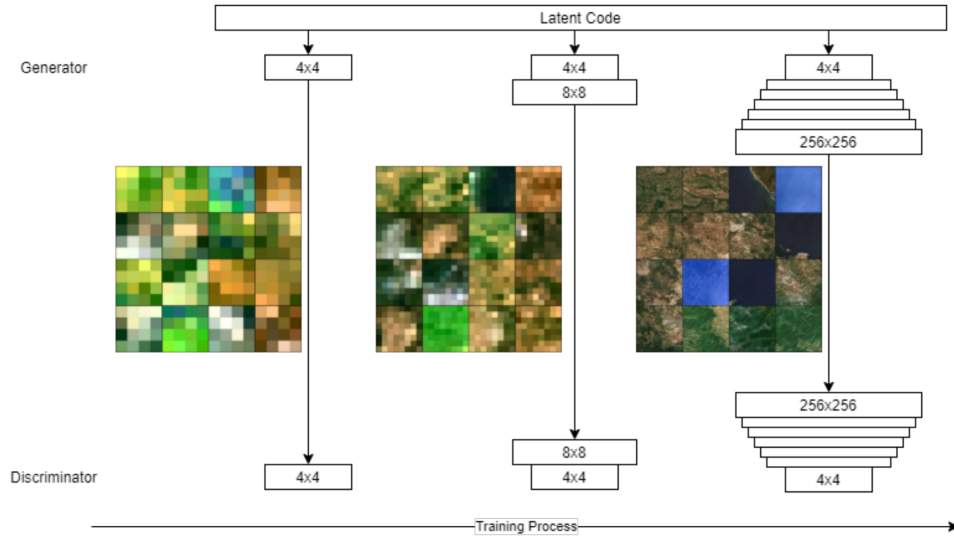


Figure 2.5: Training Process used in [25]

This diagram represents the training process. Both the Generator and Discriminator start with a low resolution of 4x4. Size is increased exponentially, until the target distribution of 256x256 is reached.

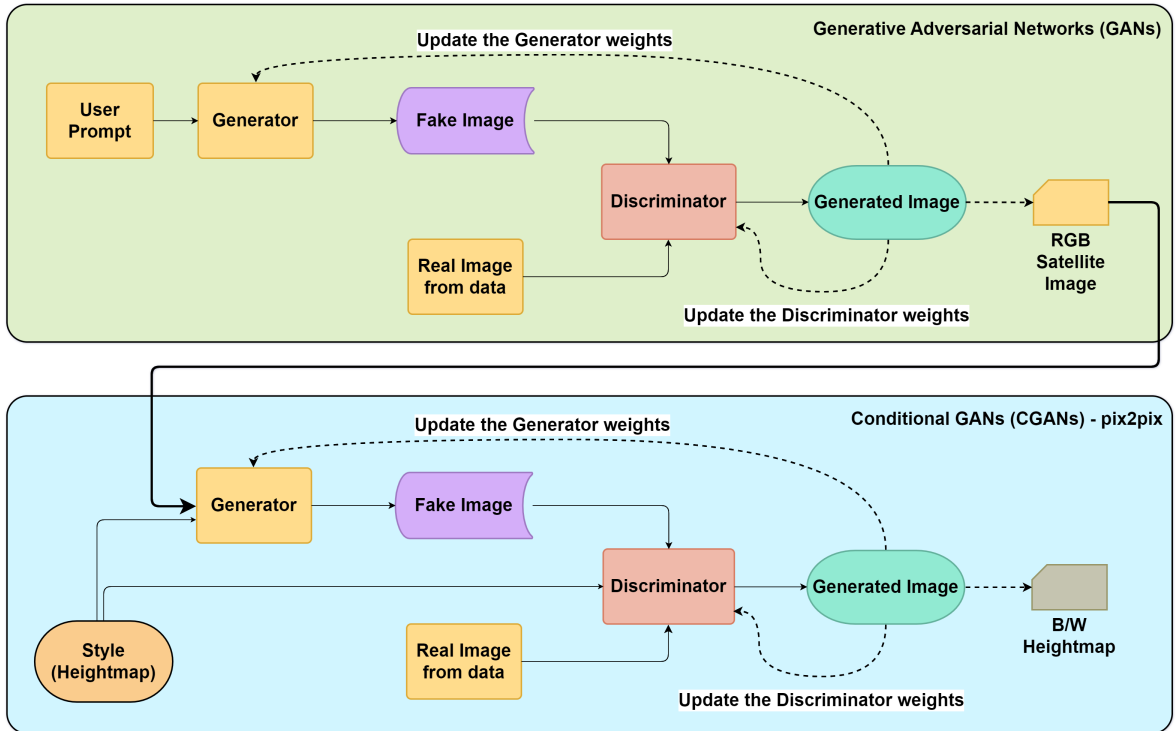


Figure 2.6: Architecture for [25]

This diagram gives an overview of the architecture followed in the reference paper. The authors are using 2 GANs, to achieve the results.

2.4.6 Interactive Guided Generation

A more recent study in [6] follows a similar approach of cascading GANs. It generates a heightfield, then iteratively increases the quality, and fine-tunes it using CGANs. The preprocessing step uses example datasets to produce *Terrain Synthesizers* [6]. A Terrain Synthesizer takes an input sketch or annotated image of a terrain and produces the output heightfield or terrain.

There are 4 different Terrain Synthesizers. A sketch-to-terrain synthesiser $\mathcal{S}$ creates terrain from a sketch containing defining elements such as rivers, altitude cues, ridges, or any combination of the 3. A levelset-to-terrain synthesiser $\mathcal{L}$ turns a binary image (monochrome) into a terrain. An eraser synthesiser $\mathcal{R}$ deletes the user-specified part of a terrain and fills the rest, with a content aware generation. The final synthesiser is an erosion synthesiser $\mathcal{E}$ that erodes the given heightfields and transforms the terrain into an eroded one. Each synthesiser here is an independent model and specialises in a specific task. Due to this, the dataset requirements are very huge and particular.

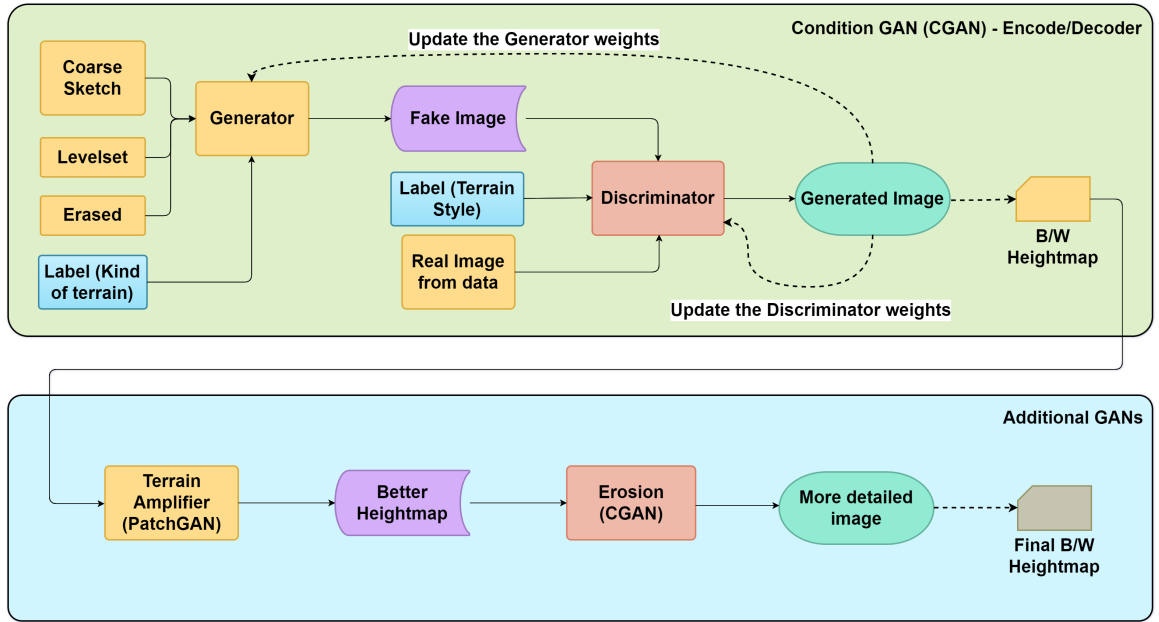


Figure 2.7: Architecture for [6]

This diagram gives an overview of the architecture followed in the reference paper. The authors use multiple GANs, which they refer to as "Terrain Synthesisers". Each GAN has a specific task that it performs and in the end the generated heightfield is upsampled using a super-sampling technique [5], and eroded using a final GAN.

An algorithm by the same authors, as mentioned in [5] is used to add small details to the generated heightfield. This is referred to as "Terrain Amplification" in [6], which is content aware and adds details based on the whole image. Figure 2.8 displays the results achieved by the authors and their sketch-based approach.

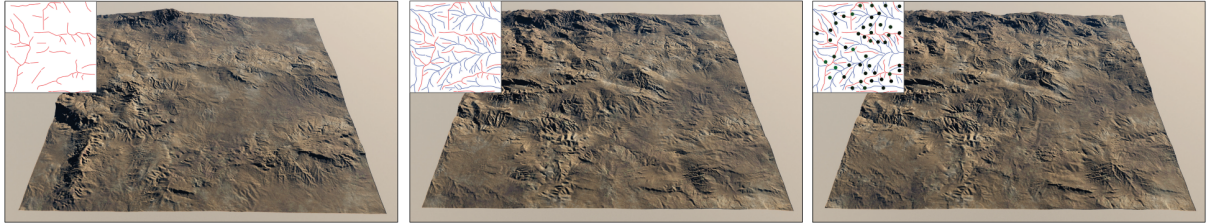


Figure 2.8: Results from [6]

The images above show results from a professional artist using the tool. He spent a few minutes to author the large terrain by using ridges (left), adjusting the terrain by adding rivers (middle), and further adding elevation points (right).

2.5 Summary

This chapter covers the various terminology necessary to understand the thesis. Furthermore, it goes over previous research done in the field of Procedural Landscape Generation using traditional and Machine Learning methods. Particularly, a deep dive into Deep Learning and its various architectures, including Generative Adversarial Networks 2.1.5, CGANs 2.1.6, and Stable Diffusion 2.2.

Heightfields 2.3 are the main focus of this thesis, similar to references including [25; 9; 6; 3]. The thesis aims at generating plausible and good quality heightfields using Stable Diffusion, which is a newer method on image generation than GANs. Not only is it faster, but also lighter and more consistent 2.2.

The next chapter covers details about the architecture and technologies used to implement this thesis. It goes over the 3D rendering pipeline, challenges faced in heightfield generation, and some details about SD and its intricacies.

Chapter 3

Methodology

This chapter covers the approach, elaborating how each component addresses the identified challenges. It highlights the pros and cons of using AI for Landscape Generation and gives an overview for the detailed application and results that are discussed in further chapters.

3.1 Problem Formulation

This section should provide the reader with an overall description of the problem that will be addressed in the dissertation. In contrast to a generic discussion of the dissertation topic in the Introduction chapter, this section should provide a detailed discussion of the problem that has been identified based on the existing work that has been discussed in the preceding chapter.

In some dissertations, it may make sense to convert this section into a short chapter of its own which follows the discussion of the existing work and precedes the discussion of the work of the dissertation.

3.1.1 Identified Challenges

The research done in this field is fairly low, but very interesting. There has been a great use of Generative Adversarial Networks (GANs), Conditions GANs (CGANs), and Patch GANs [25; 6] as mentioned in Chapter 2. Some of the identified challenges are listed below:

- Something:

3.1.2 Proposed Work

This section should provide a thorough description of the problem and an overview of the work proposed to address the problem.

3.2 Overview of the Design

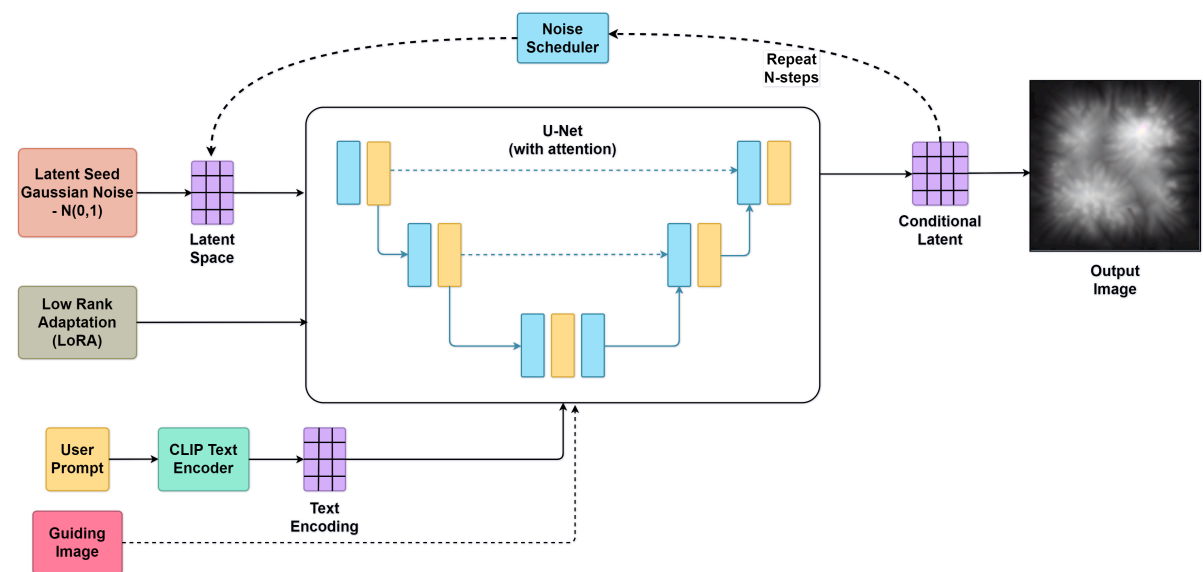


Figure 3.1: Overview of the Iterative Process in PCG

The diagram illustrates interaction between manual input and the PCG algorithm. The process starts with manual input provided by human creators, which feeds into the PCG algorithm. This algorithm generates content based on the input and iteratively refines it. Key benefits of this process include speed, reliability, controllability, diversity, and enhanced creativity and believability of the generated content.

A description of the approach that addresses the problem identified above.

3.3 Summary

Every chapter aside from the first and last chapter should conclude with a summary.

Chapter 4

Implementation

Guess what? At the beginning of each chapter, a description should introduce the reader to the content of the chapter. The description should explain to the reader the layout of the chapter, the contribution that the chapter makes to the overall dissertation and the contribution of the individual sections towards the overall chapter.

4.1 Overview of the Solution

```
x = 1
if x == 1:
    # indented four spaces
    print("x is 1.")
```

Listing 4.1: Lengthy caption explaining the code to the reader

The code in listing 4.1 is a demonstration how to include a file with code into the template.

4.2 Component One

The code in listing 4.2 is a demonstration how to include code in the template.

```
x = 1
if x == 1:
    # indented four spaces
    print("x is 1.")
```

Listing 4.2: Second Lengthy caption

4.3 Summary

Every chapter aside from the first and last chapter should conclude with a summary.

Chapter 5

Evaluation

The Evaluation chapter should present a comparison of the work that forms the basis of the dissertation and existing work. At a higher level, it should demonstrate an awareness of the relationship of the dissertation work to the research area that it is based in.

5.1 Experiments

In the case where experiments have been carried out, the experimental setup and the values that were defined for the variables need to be presented in a table e.g. table 5.1.

Column 1	Column 2	Column 3
Row 1	Item 1	Item 2
Row 2	Item 1	Item 2
Row 3	Item 1	Item 2
Row 4	Item 1	Item 2

Table 5.1: Caption that explains the table to the reader

5.2 Results

Figures that present results such as figure 5.1 need to display descriptions of the axes, the units and scales of the measurements, statistical values, etc. Where measurements were taken from experiments, error bars or confidence intervals need to be provided to give the reader an indication of the spread of the measurements.

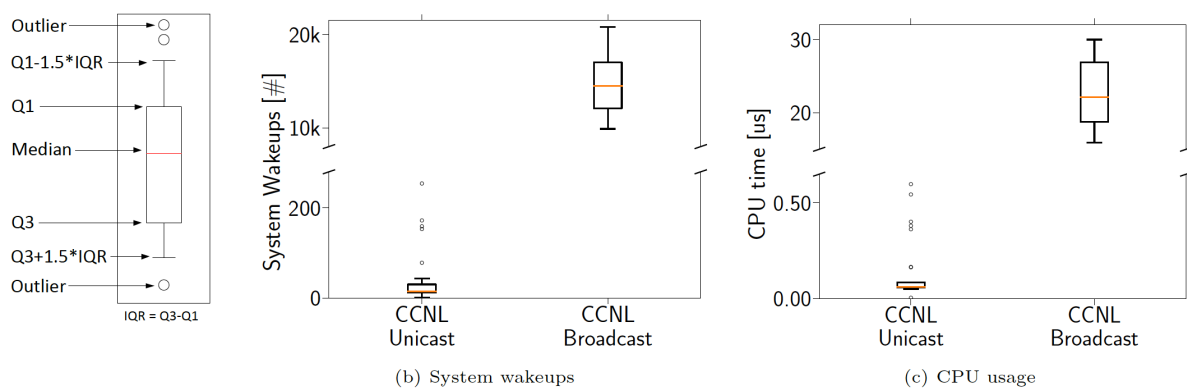


Figure 5.1: Long caption that describes the figure to the reader

5.3 Summary

Every chapter aside from the first and last chapter should conclude with a summary that presents the outcome of the chapter in a short, accessible form.

Chapter 6

Conclusions & Future Work

This chapter should summarize the work presented in the dissertation and discuss the conclusions that can be drawn from the work and the results presented in chapter 5.

6.1 Future Work

The section may present a list of items that were beyond the scope of the dissertation.

Bibliography

- [1] ALAMMAR, J. The illustrated stable diffusion, 2022.
- [2] ALZUBAIDI, L., Z. J. H. A. E. A. Review of deep learning: concepts, cnn architectures, challenges, applications, future directions. *Journal of Big Data* 8, 1 (2021), 53.
- [3] BELHADJ, F., AND AUDIBERT, P. Modeling landscapes with ridges and rivers: bottom up approach. In *Proceedings of the 3rd International Conference on Computer Graphics and Interactive Techniques in Australasia and South East Asia* (New York, NY, USA, 2005), GRAPHITE '05, Association for Computing Machinery, p. 447–450.
- [4] COMMUNITY, B. O. Blender - a 3d modelling and rendering package, 2018.
- [5] CORDONNIER, G., BRAUN, J., CANI, M.-P., BENES, B., GALIN, E., PEYTAVIE, A., AND GUÉRIN, E. Large Scale Terrain Generation from Tectonic Uplift and Fluvial Erosion. *Computer Graphics Forum* 35, 2 (May 2016), 165–175.
- [6] ERIC GUÉRIN, JULIE DIGNE, E. G. A. P. C. W. E. A. Interactive example-based terrain authoring with conditional generative adversarial networks. *ACM Transactions on Graphics* 36, 6 (2017).
- [7] FLORINSKY, I. *Digital Terrain Analysis in Soil Science and Geology*. Elsevier Science, 2016.
- [8] FOURNIER, A., FUSSELL, D., AND CARPENTER, L. Computer rendering of stochastic models. *Commun. ACM* 25, 6 (jun 1982), 371–384.
- [9] GÉNEVAUX, J.-D., GALIN, E., GUÉRIN, E., PEYTAVIE, A., AND BENES, B. Terrain generation using procedural models based on hydrology. *ACM Trans. Graph.* 32, 4 (jul 2013).
- [10] GOODFELLOW, I. J., POUGET-ABADIE, J., MIRZA, M., XU, B., WARDEFARLEY, D., OZAIR, S., COURVILLE, A., AND BENGIO, Y. Generative adversarial networks, 2014.

- [11] GUTH, P. L., VAN NIEKERK, A., GROHMANN, C. H., MULLER, J.-P., HAWKER, L., FLORINSKY, I. V., GESCH, D., REUTER, H. I., HERRERA-CRUZ, V., RIAZANOFF, S., LÓPEZ-VÁZQUEZ, C., CARABAJAL, C. C., ALBINET, C., AND STROBL, P. Digital elevation models: Terminology and definitions. *Remote Sensing* 13, 18 (2021).
- [12] HE, K., ZHANG, X., REN, S., AND SUN, J. Deep residual learning for image recognition, 2015.
- [13] HU, E. J., SHEN, Y., WALLIS, P., ALLEN-ZHU, Z., LI, Y., WANG, S., WANG, L., AND CHEN, W. Lora: Low-rank adaptation of large language models, 2021.
- [14] K. COMPTON, J. C. O., AND MATEAS, M. Generative methods. In *Proceedings of the 4th Workshop on Procedural Content Generation in Games* (2013).
- [15] KARRAS, T., AILA, T., LAINE, S., AND LEHTINEN, J. Progressive growing of gans for improved quality, stability, and variation, 2018.
- [16] KRIZHEVSKY, A., SUTSKEVER, I., AND HINTON, G. E. Imagenet classification with deep convolutional neural networks. In *Advances in neural information processing systems* (2012), pp. 1097–1105.
- [17] LEE, J. J., LI, B., AND BENES, B. Latent l-systems: Transformer-based tree generator. *ACM Trans. Graph.* 43, 1 (nov 2023).
- [18] LIN, H. C., AND BURNES, A. Introducing nvidia dlss 3. *GeForce News* (2022).
- [19] MIRZA, M., AND OSINDERO, S. Conditional generative adversarial nets, 2014.
- [20] MUSGRAVE, F. K., KOLB, C. E., AND MACE, R. S. The synthesis and rendering of eroded fractal terrains. 41–50.
- [21] NAGASHIMA, K. Computer generation of eroded valley and mountain terrains. *The Visual Computer* 13, 9–10 (Jan. 1998), 456–464.
- [22] NOOR SHAKER, J. T., AND NELSON, M. J. *Procedural Content Generation in Games*, 1st ed. Computational Synthesis and Creative Systems. Springer Cham, 2016.
- [23] OLA, F., HAN, S., MOSKEWICZ, M., ASHRAF, K., DALLY, W., AND KEUTZER, K. Squeezenet: Alexnet-level accuracy with 50x fewer parameters and 0.5 mb model size. *arXiv preprint arXiv:1602.07360* (2016).

- [24] OPENTOPOGRAPHY. OpenTopography Resources, 2023.
- [25] PANAGIOTOU, E., AND CHAROU, E. Procedural 3d terrain generation using generative adversarial networks, 2020.
- [26] PANAGIOTOU, E., CHOCHLAKIS, G., GRAMMATIKOPOULOS, L., AND CHAROU, E. Generating elevation surface from a single rgb remotely sensed image using deep learning. *Remote Sensing* 12, 12 (2020).
- [27] QUADSPINNER. Gaea, 2023.
- [28] ROMBACH, R., BLATTMANN, A., LORENZ, D., ESSER, P., AND OMMER, B. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (New Orleans, LA, June 2022), pp. 10684–10695.
- [29] RONNEBERGER, O., FISCHER, P., AND BROX, T. U-net: Convolutional networks for biomedical image segmentation, 2015.
- [30] SCHMITT, S. World machine, 2023.
- [31] SERVICES, A. W. What is gan?, 2023.
- [32] SERVICES, A. W. What is stable diffusion?, 2023.
- [33] SMITH, G. Understanding procedural content generation: a design-centric analysis of the role of pcg in games. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (New York, NY, USA, 2014), CHI '14, Association for Computing Machinery, p. 917–926.
- [34] SZEGEDY, C., LIU, W., JIA, Y., SERMANET, P., REED, S., ANGUELOV, D., ERHAN, D., VANHOUCKE, V., AND RABINOVICH, A. Going deeper with convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (2015), pp. 1–9.
- [35] TAIT, E. R., AND NELSON, I. L. Nonscalability and generating digital outer space natures in no man’s sky. *Environment and Planning E: Nature and Space* 5, 2 (2022), 694–718.
- [36] TECHTARGET. What is a convolutional neural network?, 2023.
- [37] TOGELIUS, J., K. E. S. D., AND YANNAKAKIS, G. N. What is procedural content generation?: Mario on the borderline. In *Proceedings of the 2nd Workshop on Procedural Content Generation in Games* (2011).

- [38] VAN MUIJDEN, J. Gpu-based procedural placement in horizon zero dawn. In *Proceedings of the Game Developers Conference (GDC)* (2017).
- [39] ZHOU, H., SUN, J., TURK, G., AND REHG, J. M. Terrain synthesis from digital elevation models. *IEEE Transactions on Visualization and Computer Graphics* 13, 4 (July 2007), 834–848.
- [40] ZIOMA, R. *GPU-Generated Procedural Wind Animations for Trees*. Addison-Wesley Professional, 2007.